

(4) statsmodel

Part 1 Linear regression: Modelling time spent at the hospital

- We will perform a regression analysis to model the number of days spent at the hospital, among the population of patients.
- To get started with our model, we need two components:
 1. The equation describing the model
 2. The data
- Equations are specified using patsy formula syntax. Important operators are:
 1. `~` : Separates the left-hand side and right-hand side of a formula.
 2. `+` : Creates a union of terms that are included in the model.
 3. `:` : Interaction term.
 4. `*` : `a * b` is short-hand for `a + b + a:b` , and is useful for the common case of wanting to include all interactions between a set of variables.
- Intercepts are added by default.
- Categorical variables can be included directly by adding a term `C(a)`. More on that soon!
- For (2), we can conveniently use pandas dataframe.

An example

- Let's start with an example from our dataset. We are interested in two predictors: diabetes and high blood pressure. These are the two predictors that we want to use to fit the outcome, the number of days spent at the hospital, using a linear regression.
- A model that achieves this is formulated as:
`time ~ C(diabetes) + C(high_blood_pressure)`
- We can create this model using `smf.ols()`.
- OLS stands for ordinary least squares linear regression.
- The two components: the formula and the data are stated explicitly.
- The terms in the formula are columns in pandas dataframe. Easy!

```
# Declares the model
mod = smf.ols(formula='time ~ C(diabetes) + C(high_blood_pressure)',
data=df)

# Fits the model (find the optimal coefficients, adding a random seed
ensures consistency)
np.random.seed(2)
res = mod.fit()
```

```
# Print the summary output provided by the library.  
print(res.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          time    R-squared:
0.040
Model:                  OLS     Adj. R-squared:
0.033
Method:                 Least Squares    F-statistic:
6.097
Date:                  Fri, 04 Oct 2024    Prob (F-statistic):
0.00254
Time:                  16:16:09    Log-Likelihood:
-1718.9
No. Observations:      299    AIC:
3444.
Df Residuals:          296    BIC:
3455.
Df Model:               2
Covariance Type:       nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|
[0.025    0.975]
-----
Intercept                139.3851      6.658     20.934     0.000
126.282    152.489
C(diabetes) [T.1]         4.9059      8.949      0.548     0.584
-12.706     22.518
C(high_blood_pressure) [T.1] -31.8228      9.247     -3.441     0.001
-50.021    -13.624
=====
```

```
=====
Omnibus:                159.508    Durbin-Watson:
0.076
Prob(Omnibus):          0.000    Jarque-Bera (JB):
18.166
Skew:                   0.076    Prob(JB):
0.000114
Kurtosis:               1.802    Cond. No.
2.82
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is

correctly specified.

A lot of useful information is provided by default.

- The dependent variable : time (number of days at the hospital)
- Method: The type of model that was fitted (OLS)
- Nb observations: The number of datapoints (299 patients)
- R2: The fraction of explained variance
- A list of predictors
- For each predictor: coefficient, standard error of the coefficients, p-value, 95% confidence intervals. We can see that only high blood pressure is a significant predictor ($p = 0.001$), while diabetes is not (0.584).
- Warnings if there are numerical issues (hopefully not!)

We interpret the model in the following way: days at hospital = $139 + 4.9 \text{ diabetes} - 31.8 \text{ high blood pressure}$.

```
# we use a*b to add terms: a, b, a:b, and intercept

mod = smf.ols(formula='time ~ C(high_blood_pressure) * C(DEATH_EVENT,
Treatment(reference=0)) + C(diabetes)',
              data=df)

res = mod.fit()
```

Logistic regression: Modelling the binary death outcome

Standardize continuous variables

```
# how we standardize the countinuous variables
df['age'] = (df['age'] - df['age'].mean())/df['age'].std()
df['creatinine_phosphokinase'] = (df['creatinine_phosphokinase'] -
df['creatinine_phosphokinase'].mean())/df['creatinine_phosphokinase'].std(
)
df['ejection_fraction'] = (df['ejection_fraction'] -
df['ejection_fraction'].mean())/df['ejection_fraction'].std()
df['platelets'] = (df['platelets'] -
df['platelets'].mean())/df['platelets'].std()
df['serum_creatinine'] = (df['serum_creatinine'] -
df['serum_creatinine'].mean())/df['serum_creatinine'].std()
df['serum_sodium'] = (df['serum_sodium'] -
df['serum_sodium'].mean())/df['serum_sodium'].std()
```

Fit the model

```
# logit is logistic regression. The other parameters are the same as
before

mod = smf.logit(formula='DEATH_EVENT ~ age + creatinine_phosphokinase +
ejection_fraction + \
                    platelets + serum_creatinine + serum_sodium + \
                    C(diabetes) + C(high_blood_pressure) + \
                    C(sex) + C(anaemia) + C(smoking) +
C(high_blood_pressure)', data=df)
res = mod.fit()
print(res.summary())
```

```
# feature names
variables = res.params.index

# quantifying uncertainty!

# coefficients
coefficients = res.params.values

# p-values
p_values = res.pvalues

# standard errors
standard_errors = res.bse.values

#confidence intervals
res.conf_int()
```

Now we can visualize the effect of all the predictors. Let's first sort them by the coefficients.

```
# sort them all by coefficients
l1, l2, l3, l4 = zip(*sorted(zip(coefficients[1:], variables[1:],
standard_errors[1:], p_values[1:])))

# in this case, we index starting from the first element, not to plot the
intercept

# we will use standard errors, instead of CIs
# two standard errors approximate the CIs (you can actually see in the
summary table that
# +/-2 SE is equivalent to the CIs)
```

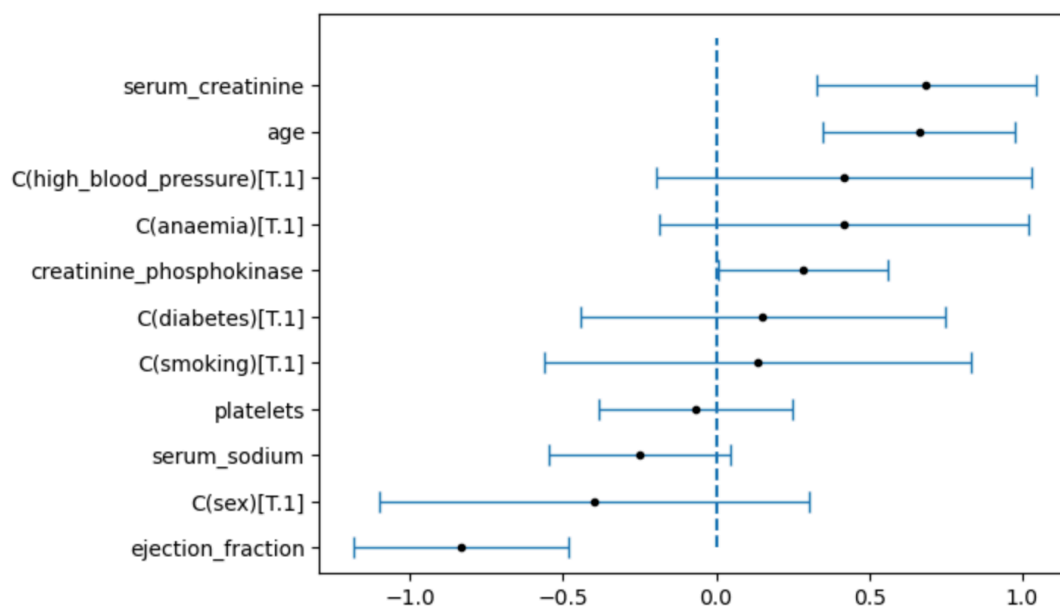
Plot

```
# fancy plotting

plt.errorbar(l1, np.array(range(len(l1))), xerr= 2*np.array(l3), linewidth
= 1,
            linestyle = 'none',marker = 'o',markersize= 3,
            markerfacecolor = 'black',markeredgcolor = 'black', capsiz=
5)

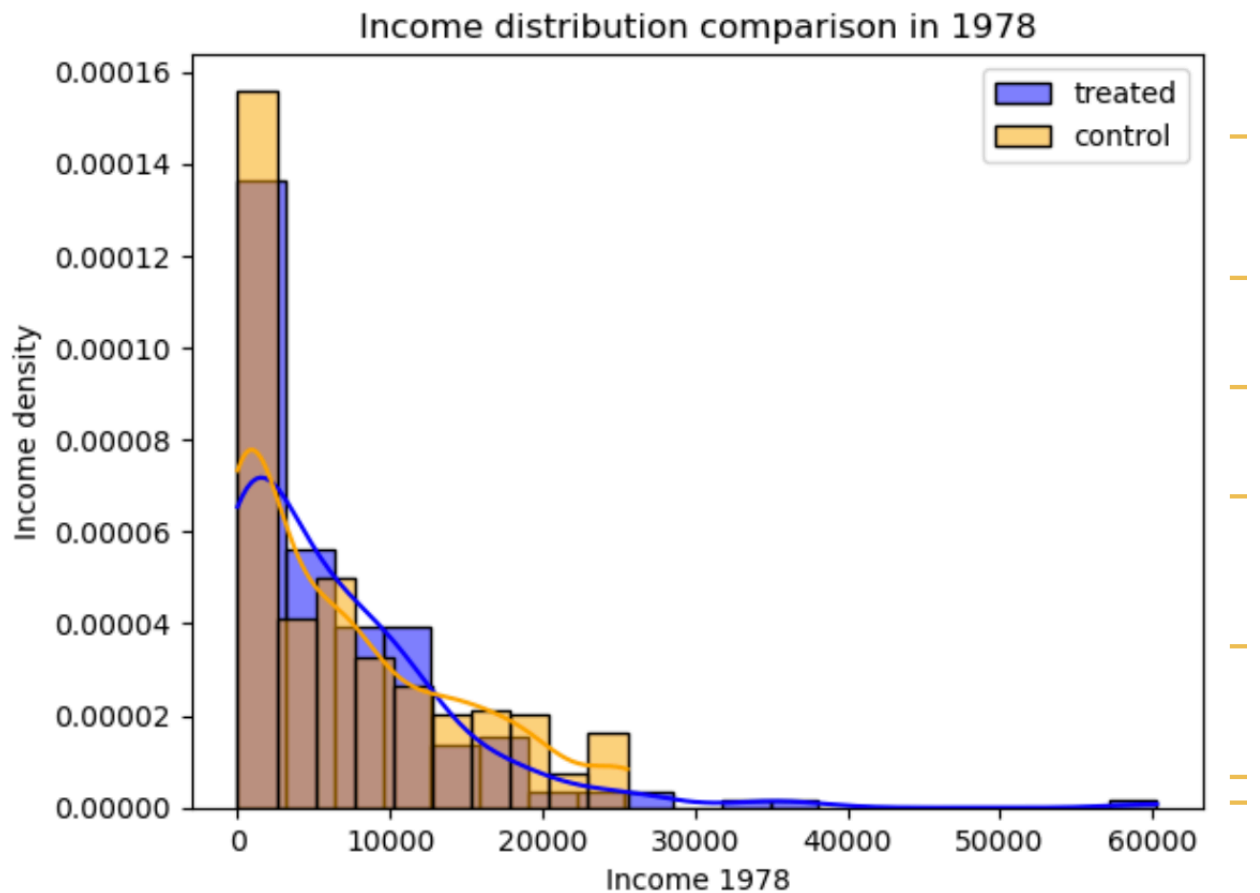
plt.vlines(0,0, len(l1), linestyle = '--')

plt.yticks(range(len(l2)),l2);
plt.show()
```



```
''' your code and explanations ''';
treated = df.loc[df['treat'] == 1] #People that attained the programme
control = df.loc[df['treat'] == 0] #People that didn't attain the
programme

ax = sns.histplot(treated['re78'], kde=True, stat='density', color='blue',
label='treated')
ax = sns.histplot(control['re78'], kde=True, stat='density',
color='orange', label='control')
ax.set(title='Income distribution comparison in 1978',xlabel='Income
1978', ylabel='Income density')
plt.legend()
plt.show()
```



Propensity matching (naive)

```
# let's standardize the continuous features
lalonge_data['age'] = (lalonge_data['age'] -
lalonge_data['age'].mean())/lalonge_data['age'].std()
lalonge_data['educ'] = (lalonge_data['educ'] -
lalonge_data['educ'].mean())/lalonge_data['educ'].std()
lalonge_data['re74'] = (lalonge_data['re74'] -
lalonge_data['re74'].mean())/lalonge_data['re74'].std()
lalonge_data['re75'] = (lalonge_data['re75'] -
lalonge_data['re75'].mean())/lalonge_data['re75'].std()

mod = smf.logit(formula='treat ~ age + educ + C(black) + C(hispan) +
C(married) + C(nodgree) + \
+re74 + re75', data=lalonge_data)

res = mod.fit()

# Extract the estimated propensity scores
lalonge_data['Propensity_score'] = res.predict()

print(res.summary())
```

Use the propensity scores to match each data point from the treated group with exactly one data point from the control group, while ensuring that each data point from the control group is matched with at most one data point from the treated group.

(Hint: you may explore the `networkx` package in Python for predefined matching functions.) Your matching should maximize the similarity between matched subjects, as captured by their propensity scores.

In other words, the sum (over all matched pairs) of absolute propensity-score differences between the two matched subjects should be minimized.

After matching, you have as many treated as you have control subjects.

Compare the outcomes (`re78`) between the two groups (treated and control).

For this task, we implement the simplest, full optimal matching, and analyse the results.

In order to perform matching between pairs, a distance/similarity function is needed. Since the library used for the matching (`networkx`) has a function that maximizes the sum of weights between pairs, it is natural to use a function that measures similarity.

Wanting to minimize the difference of propensity scores between pairs, we define the similarity function between two instances x and y like:

$$\text{similarity}(x, y) = 1 - |\text{propensity_score}(x) - \text{propensity_score}(y)|$$

This function captures the difference in scores like a distance, but since the distance defined like that would always be between 0 and 1, subtracting it from 1 would be a meaningful measure to use for similarity.

```
def get_similarity(propensity_score1, propensity_score2):
    '''Calculate similarity for instances with given propensity scores'''
    return 1-np.abs(propensity_score1-propensity_score2)

# Separate the treatment and control groups
treatment_df = lalonde_data[lalonde_data['treat'] == 1]
control_df = lalonde_data[lalonde_data['treat'] == 0]

# Create an empty undirected graph
G = nx.Graph()

# Loop through all the pairs of instances
for control_id, control_row in control_df.iterrows():
    for treatment_id, treatment_row in treatment_df.iterrows():

        # Calculate the similarity
        similarity = get_similarity(control_row['Propensity_score'],
                                    treatment_row['Propensity_score'])

        # Add an edge between the two instances weighted by the similarity
        # between them
        G.add_weighted_edges_from([(control_id, treatment_id,
```

```

similarity)))

# Generate and return the maximum weight matching on the generated graph
matching = nx.max_weight_matching(G)

matched = [i[0] for i in list(matching)] + [i[1] for i in list(matching)]
balanced_df_1 = lalonde_data.iloc[matched]

```

Better matching

```

treatment_df = lalonde_data[lalonde_data['treat'] == 1]
control_df = lalonde_data[lalonde_data['treat'] == 0]

G = nx.Graph()

for control_id, control_row in control_df.iterrows():
    for treatment_id, treatment_row in treatment_df.iterrows():

        # Adds an edge only if the individuals have the same race
        if (control_row['black'] == treatment_row['black'])\
            and (control_row['hispan'] == treatment_row['hispan']):
            similarity = get_similarity(control_row['Propensity_score'],
                                      treatment_row['Propensity_score'])

            G.add_weighted_edges_from([(control_id, treatment_id,
similarity)])

matching = nx.max_weight_matching(G)

matched = [i[0] for i in list(matching)] + [i[1] for i in list(matching)]

balanced_df_all = lalonde_data.iloc[matched]

```

Analyse with the balanced data

```

treated = balanced_df_all.loc[balanced_df_all['treat'] == 1] #People that
attained the program
control = balanced_df_all.loc[balanced_df_all['treat'] == 0] #People that
didn't attain the program

treated.re78.describe()
control.re78.describe()

ax = sns.histplot(treated['re78'], kde=True, stat='density', color='blue',
label='treated');
ax = sns.histplot(control['re78'], kde=True, stat='density',
color='orange', label='control')

```



```
ax.set(title='Income distribution comparison in 1978, after  
matching',xlabel='Income 1978', ylabel='Income density')  
plt.legend()  
plt.show()
```

Final conclusion: after the propensity score matching, the results drastically change and support the # positive effect of the training program.

